



SAVEETHA SCHOOL OF ENGINEERING



SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

NAME : G. DHANUSH

Course Code:CSA1709

REG NO : 192512132

Course Name:Artificial Intelligence

17.

Aim: To write a Prolog program to calculate the sum of the first 'n' integers.

Algorithm:

- **Step 1:** Define the base case fact where the sum of 0 is 0 (sum(0,0).).
- **Step 2:** Define the recursive rule sum(N,S) that applies when the integer N is greater than 0.
- **Step 3:** Calculate the previous integer N1 by subtracting 1 from N (N1 is N - 1).
- **Step 4:** Recursively call the sum predicate using N1 to find its corresponding sum S1 (sum(N1,S1)).
- **Step 5:** Calculate the final sum S by adding N to S1 (S is S1 + N).

PROGRAM:

```
/* sum of n integers */
```

```
sum(0,0).
```

```
sum(N,S) :-
```

```
    N > 0,
```

```
    N1 is N -1,
```

```
    sum(N1,S1),
```

```
    S is S1 + N.
```

OUTPUT:



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/DELL/OneDrive/Desktop/Documents/sum of n integers.pl').
compiling C:/Users/DELL/OneDrive/Desktop/Documents/sum of n integers.pl for byte code...
C:/Users/DELL/OneDrive/Desktop/Documents/sum of n integers.pl compiled, 8 lines read - 866 bytes written, 12 m
yes
| ?- sum(10,X).

X = 55 ? ;
no
| ?- sum(5,Answer).

Answer = 15 ? ;
(15 ms) no
| ?- |
```

Result: Thus, the program was executed and found that the output was found to be correct.

18. BIRTHDAY

Aim:

To write a Prolog program to establish a database of birthdays and execute queries regarding birth years, birth months, and age comparisons.

Algorithm:

- **Step 1:** Define the database facts specifying the date of birth for various individuals using the format `dob(Name, Year, Month, Day)`.
- **Step 2:** Create Rule 1 (`born_in_year`) to find individuals born in a specific year by extracting the Year variable from the `dob` fact.
- **Step 3:** Create Rule 2 (`born_in_month`) to find individuals born in a specific month by extracting the Month variable from the `dob` fact.
- **Step 4:** Create Rule 3 (`born_before`) to compare the birth years of two distinct people to check if the first person's birth year is less than the second person's.
- **Step 5:** Create Rule 4 (`same_birth_year`) to identify pairs of different individuals who share the exact same birth year.

PROGRAM:

```
% FACTS: dob(Name, Year, Month, Day)
```

```
dob(alice, 1995, 5, 12).
```

```
dob(bob, 2001, 11, 23).
```

```
dob(charlie, 1988, 5, 5).
```

```
dob(david, 1995, 8, 30).
```

```
dob(eve, 2005, 2, 14).
```

```
dob(frank, 1988, 12, 1).
```

```
% RULES
```

```
% Rule 1: Find people born in a specific year
```

```
born_in_year(Name, Year) :-
```

dob(Name, Year, _, _).

% Rule 2: Find people born in a specific month (1 = Jan, 2 = Feb, etc.)

born_in_month(Name, Month) :-

dob(Name, _, Month, _).

% Rule 3: Check if Person1 was born in an earlier year than Person2

born_before(Person1, Person2) :-

dob(Person1, Year1, _, _),

dob(Person2, Year2, _, _),

Year1 < Year2.

% Rule 4: Find pairs of people who were born in the same year

same_birth_year(Person1, Person2) :-

dob(Person1, Year, _, _),

dob(Person2, Year, _, _),

Person1 \= Person2.

OUTPUT:



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul 8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/DELL/OneDrive/Desktop/Documents/bi.pl'),
uncaught exception: error(existence_error(source_sink, 'C:/Users/DELL/OneDrive/Desktop/Documents/bi.pl'), consult/1)
| ?- consult('C:/Users/DELL/OneDrive/Desktop/Documents/birthdays.pl').
compiling C:/Users/DELL/OneDrive/Desktop/Documents/birthdays.pl for byte code...
C:/Users/DELL/OneDrive/Desktop/Documents/birthdays.pl compiled. 29 lines read - 2303 bytes written; 46 ms

yes
| ?- dob(alice, Year, Month, Day).
Day = 12
Month = 5
Year = 1995

yes
| ?- born_in_year(Person, 1988).
Person = charlie ? ;
Person = frank

yes
| ?- born_before(charlie, bob).
yes
| ?- |
```

Result: Thus, the program was executed and found that the output was found to be correct.

19. STUDENT TEACHER DATABASE

Aim:

To write a Prolog program that models a student-teacher relational database to extract information about course enrollments, instructors, and classmates.

Algorithm:

- Step 1: Define facts mapping teachers to the specific subject codes they teach (teaches(Teacher, SubjectCode)).
- Step 2: Define facts mapping students to the specific subject codes they are enrolled in (enrolled(Student, SubjectCode)).
- Step 3: Define facts associating subject codes with their full subject names (subject(SubjectCode, SubjectName)).
- Step 4: Create Rule 1 (teacher_of) to establish that a teacher teaches a student if they both share the same subject code.
- Step 5: Create Rule 2 (student_in_class) to retrieve students taught by a specific teacher for a specific subject code.
- Step 6: Create Rule 3 (classmates_by_teacher) to verify if two distinct students are taught by the same teacher.
- Step 7: Create Rule 4 (student_course_details) to consolidate full course information, retrieving the student, subject code, subject name, and teacher simultaneously.

PROGRAM:

```
% teaches(Teacher, SubjectCode)
teaches(dr_smith, cs101).
teaches(dr_smith, cs102).
teaches(prof_jones, math201).
teaches(prof_davis, phys101).
teaches(dr_wilson, cs101).
```

```
% enrolled(Student, SubjectCode)
```

```
enrolled(alice, cs101).
```

```
enrolled(alice, math201).
```

```
enrolled(bob, cs102).
```

```
enrolled(charlie, phys101).
```

```
enrolled(charlie, cs101).
```

```
enrolled(david, math201).
```

```
% subject(SubjectCode, SubjectName)
```

```
subject(cs101, 'Introduction to Computer Science').
```

```
subject(cs102, 'Data Structures').
```

```
subject(math201, 'Calculus II').
```

```
subject(phys101, 'General Physics I').
```

% Rule 1: A teacher teaches a student if they teach a subject code the student is enrolled in.

```
teacher_of(Teacher, Student) :-
```

```
    teaches(Teacher, SubjectCode),
```

```
    enrolled(Student, SubjectCode).
```

% Rule 2: Find all students taught by a specific teacher for a specific subject code.

```
student_in_class(Student, Teacher, SubjectCode) :-
```

```
    teaches(Teacher, SubjectCode),
```

```
    enrolled(Student, SubjectCode).
```

% Rule 3: Check if two students share the same teacher.

classmates_by_teacher(Student1, Student2, Teacher) :-

teacher_of(Teacher, Student1),

teacher_of(Teacher, Student2),

Student1 \= Student2.

% Rule 4: Get full course info for a student (Student, SubjectCode, SubjectName, Teacher)

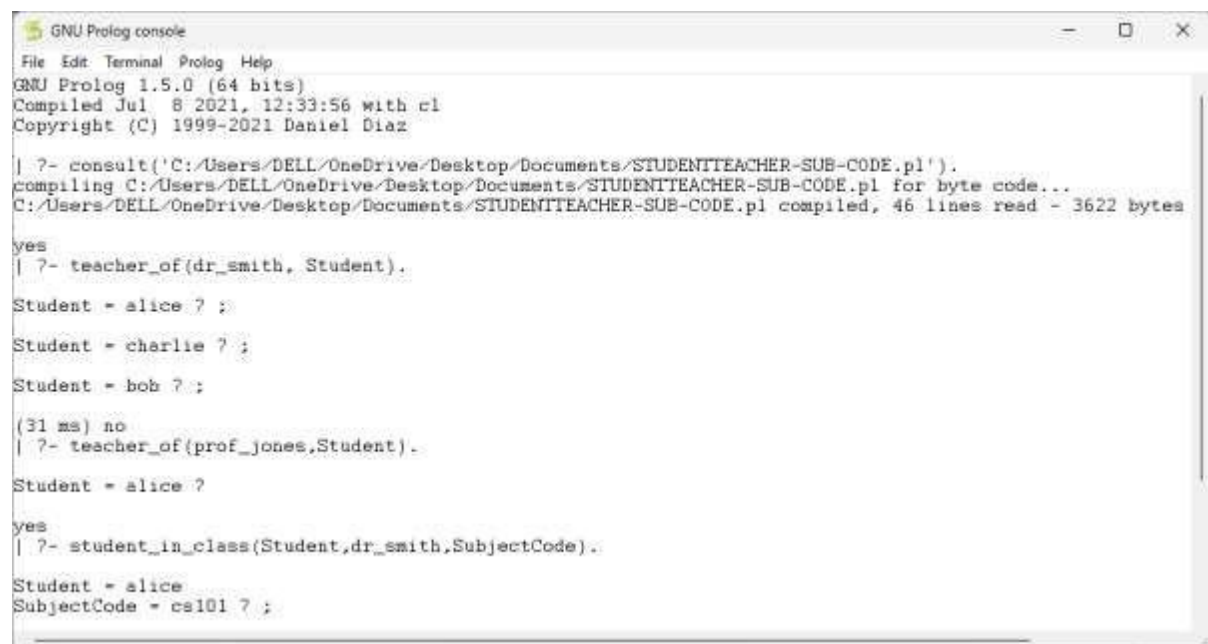
student_course_details(Student, SubjectCode, SubjectName, Teacher) :-

enrolled(Student, SubjectCode),

subject(SubjectCode, SubjectName),

teaches(Teacher, SubjectCode).

OUTPUT:



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/DELL/OneDrive/Desktop/Documents/STUDENTTEACHER-SUB-CODE.pl').
compiling C:/Users/DELL/OneDrive/Desktop/Documents/STUDENTTEACHER-SUB-CODE.pl for byte code...
C:/Users/DELL/OneDrive/Desktop/Documents/STUDENTTEACHER-SUB-CODE.pl compiled, 46 lines read - 3622 bytes

yes
| ?- teacher_of(dr_smith, Student).

Student = alice ? ;
Student = charlie ? ;
Student = bob ? ;

(31 ms) no
| ?- teacher_of(prof_jones, Student).

Student = alice ?

yes
| ?- student_in_class(Student, dr_smith, SubjectCode).

Student = alice
SubjectCode = cs101 ? ;
```

Result: Thus, the program was executed and found that the output was found to be correct.

20. PLANETS DB

Aim:

To write a Prolog program to structure a database of planetary characteristics and query them based on attributes like rings, moons, distance, and classification.

Algorithm:

- Step 1: Define facts for various planets detailing their name, classification type, distance from the sun in millions of kilometers, number of moons, and presence of rings (planet_info).
- Step 2: Create Rule 1 (has_rings) to filter and find planets where the ring attribute is listed as 'yes'.
- Step 3: Create Rule 2 (has_moons) to filter and find planets where the recorded number of moons is strictly greater than 0.
- Step 4: Create Rule 3 (planet_type) to query and categorize planets by their specific classification type (e.g., terrestrial, gas_giant).
- Step 5: Create Rule 4 (closer_to_sun) to compare the distances of two given planets and determine if the first has a smaller distance than the second.
- Step 6: Create Rule 5 (many_moons) to identify heavily moon-rich planets by checking if their moon count exceeds 10.

PROGRAM:

% FACTS: planet_info(Name, Type, DistanceFromSun_Million_Km, Moons, HasRings)

planet_info(mercury, terrestrial, 58, 0, no).

planet_info(venus, terrestrial, 108, 0, no).

planet_info(earth, terrestrial, 150, 1, no).

planet_info(mars, terrestrial, 228, 2, no).

planet_info(jupiter, gas_giant, 778, 95, yes).

planet_info(saturn, gas_giant, 1427, 146, yes).

planet_info(uranus, ice_giant, 2871, 27, yes).

planet_info(neptune, ice_giant, 4497, 14, yes).

% Rule 1: Find planets that have rings

has_rings(Planet) :-

planet_info(Planet, _, _, yes).

% Rule 2: Find planets that have at least one moon

has_moons(Planet) :-

planet_info(Planet, _, _, Moons, _),

Moons > 0.

% Rule 3: Find planets by their classification type (e.g., terrestrial, gas_giant)

planet_type(Planet, Type) :-

planet_info(Planet, Type, _, _, _).

% Rule 4: Check if Planet1 is closer to the sun than Planet2

closer_to_sun(Planet1, Planet2) :-

planet_info(Planet1, _, Dist1, _, _),

planet_info(Planet2, _, Dist2, _, _),

Dist1 < Dist2.

% Rule 5: Find heavily heavily cratered/moon-rich planets (more than 10 moons)

many_moons(Planet) :-

planet_info(Planet, _, _, Moons, _),

Moons > 10.

OUTPUT:



```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/DELL/OneDrive/Desktop/Documents/planets.pl').
compiling C:/Users/DELL/OneDrive/Desktop/Documents/planets.pl for byte code...
C:/Users/DELL/OneDrive/Desktop/Documents/planets.pl compiled, 35 lines read - 3348 bytes written, 39 ms

(15 ms) yes
| ?- has_rings(Planet)..

Planet = jupiter ? ;
Planet = saturn ? ;
Planet = uranus ? ;
Planet = neptune

(187 ms) yes
| ?- planet_info(jupiter, _, _, Moons, _).

Moons = 95

yes
| ?-
```

Result: Thus, the program was executed and found that the output was found to be correct.